respond with a success message (or did not respond in time). At this point, the request might automatically retry or report an error and have you retry manually, depending on the logic implemented in the client… And now you have two copies of your message on the server. This happens because the different HTTP methods for changing resources (`POST`, `PUT`, `DELETE`, and `PATCH`) vary in their idempotence: an idempotent operation is simply one that can be repeated many times without any subsequent change in state beyond the initial change. `POST` requests are not idempotent and should therefore result in a change in state every time they are made regardless of the data, and this is where we get the multiple comments/forum posts from when web apps have slight hiccups.

As noted earlier, `PUT` requests will contain all the data necessary to create a new resource or fully replace an existing one. Therefore, making the same `PUT` request repeatedly should have the same effect as making it (successfully) once, and such a request is said to be idempotent. `GET`, `HEAD`, `OPTIONS`, and `TRACE` are all technically idempotent too (and indeed these four methods are also said to be "safe" because they should never result in any data changing on the server). `DELETE` requests are idempotent because you can only delete a specific resource once. On the other hand, `POST` and `PATCH` requests are not idempotent, because they do not replace entire resources.

When considering if the request your application handles should be `POST`, `PUT`, etc., consider what would happen if the request was replayed with identical data. If the second request should result in a different state for, say, entities in your database, then the request is not idempotent, and you should go with `POST` (or `PATCH`).

Resource Paths and Routing

Now, I appreciate that the idea of idempotent methods and choosing the most correct HTTP method for the operation might not make much sense right now. So far we've been considering resource paths in terms of them mapping directly to different files that can be read by the web server: `GET /index.php` causes the web server (PHP's built-in `cli-server` in the case of this chapter so far) to search for a file called `index.php` in some part of the server's filesystem, read (or perhaps execute via CGI binary) the contents, and shove the results into the request body. `POST /subdir/entity.php` has the server traversing the filesystem to the `subdir` sub-directory and looking for the file `entity.php`… and so on.

This one-to-one mapping of resource paths in requests to files on a disk is fine for small projects, but web servers today are capable of far more complexity, and it is more common these days to logically decouple the URI from the file where all the magic